Object Oriented Programming (CSEG1044)

Miscellaneous: Why Do We Need Interfaces?

Tofik Ali

School of Computer Science, UPES Dehradun

February 26, 2026

Sources: Schildt, *Java: The Complete Reference* (12e) (interfaces).; Bloch, *Effective Java* (3e) (prefer interfaces where appropriate).; Oracle Java Tutorials: Interfaces (accessed February 26, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 26, 2026).

Quick Links

Overview Motivation Use Cases Summary

Agenda

1. Overview
2. Motivation (Tight Coupling)
3. Contract (Interface)
4. Use Cases
5. Java Examples
6. Summary

Learning Outcomes

- Explain what an interface represents (a contract for behavior).

Learning Outcomes

- Explain what an interface represents (a contract for behavior).
- Explain why interfaces reduce coupling and improve design flexibility.

Learning Outcomes

- Explain what an interface represents (a contract for behavior).
- Explain why interfaces reduce coupling and improve design flexibility.
- Apply interfaces in common scenarios: strategy, testing, and data access.

Learning Outcomes

- Explain what an interface represents (a contract for behavior).
- Explain why interfaces reduce coupling and improve design flexibility.
- Apply interfaces in common scenarios: strategy, testing, and data access.
- Decide when to use an interface vs an abstract class (rule-of-thumb).

The one-line answer

Why do we need interfaces?

So our code depends on **contracts** (behavior) rather than **concrete classes** (implementation).

- Easier to change (swap implementations without rewriting business logic)

The one-line answer

Why do we need interfaces?

So our code depends on **contracts** (behavior) rather than **concrete classes** (implementation).

- Easier to change (swap implementations without rewriting business logic)
- Easier to extend (add new implementations without touching existing code)

The one-line answer

Why do we need interfaces?

So our code depends on **contracts** (behavior) rather than **concrete classes** (implementation).

- Easier to change (swap implementations without rewriting business logic)
- Easier to extend (add new implementations without touching existing code)
- Easier to test (replace real dependencies with fakes/mocks)

Without interfaces: a rigid design

```
class CardPayment {
    void pay(int amount) { /* call bank */ }
}

class CheckoutService {
    void checkout(int amount) {
        CardPayment payment = new CardPayment(); // fixed
            choice
        payment.pay(amount);
    }
}
```

Symptoms of tight coupling

- New requirement (UPI/Wallet) forces edits in `CheckoutService`

Symptoms of tight coupling

- New requirement (UPI/Wallet) forces edits in `CheckoutService`
- Testing is painful (real bank calls, slow and flaky)

Symptoms of tight coupling

- New requirement (UPI/Wallet) forces edits in `CheckoutService`
- Testing is painful (real bank calls, slow and flaky)
- High-level business code depends on low-level implementation details

Introduce a contract

```
interface PaymentMethod {
    void pay(int amount);
}

class CardPayment implements PaymentMethod {
    public void pay(int amount) { /* call bank */ }
}

class UpiPayment implements PaymentMethod {
    public void pay(int amount) { /* call UPI */ }
}
```

Business logic depends on the contract

```
class CheckoutService {
    private final PaymentMethod payment;

    CheckoutService(PaymentMethod payment) {
        this.payment = payment;
    }

    void checkout(int amount) {
        payment.pay(amount);
    }
}
```

What we gained

- **Flexibility:** choose implementation at runtime (Card/UPI/Wallet)

`CheckoutService` → `PaymentMethod` ← `CardPayment` / `UpiPayment`

What we gained

- **Flexibility:** choose implementation at runtime (Card/UPI/Wallet)
- **Extensibility:** add a new payment type without changing checkout

`CheckoutService` → `PaymentMethod` ← `CardPayment` / `UpiPayment`

What we gained

- **Flexibility:** choose implementation at runtime (Card/UPI/Wallet)
- **Extensibility:** add a new payment type without changing checkout
- **Testability:** pass a fake implementation in tests

`CheckoutService` → `PaymentMethod` ← `CardPayment` / `UpiPayment`

Use case 1: plug-in behavior (Strategy)

```
interface DiscountPolicy { int discount(int total); }
class NoDiscount implements DiscountPolicy {
    public int discount(int total) { return 0; }
}
class StudentDiscount implements DiscountPolicy {
    public int discount(int total) { return total / 10; }
        // 10%
}
class Billing {
    private final DiscountPolicy policy;
    Billing(DiscountPolicy policy) { this.policy = policy;
         }
    int finalAmount(int total) { return total - policy.
        discount(total); }
}
```

Use case 2: unit testing with a fake

```
interface Notifier { void send(String to, String msg); }

class FakeNotifier implements Notifier {
    String lastTo, lastMsg;
    public void send(String to, String msg) {
        lastTo = to; lastMsg = msg;
    }
}
```

- Real code uses `EmailNotifier`; tests use `FakeNotifier`

Use case 2: unit testing with a fake

```
interface Notifier { void send(String to, String msg); }

class FakeNotifier implements Notifier {
    String lastTo, lastMsg;
    public void send(String to, String msg) {
        lastTo = to; lastMsg = msg;
    }
}
```

- Real code uses `EmailNotifier`; tests use `FakeNotifier`
- No real network calls; tests become fast and reliable

Use case 3: decouple storage (Repository)

```
class User { String id; String name; }

interface UserRepository {
    User findById(String id);
    void save(User u);
}

class InMemoryUserRepository implements UserRepository {
    /* for tests */ }
class SqlUserRepository implements UserRepository { /* for
     production */ }
```

Interfaces you already use in Java

- Collections: `List`, `Set`, `Map`, `Iterable`, `Iterator`

Interfaces you already use in Java

- Collections: `List`, `Set`, `Map`, `Iterable`, `Iterator`
- Ordering: `Comparable`, `Comparator`

Interfaces you already use in Java

- Collections: `List`, `Set`, `Map`, `Iterable`, `Iterator`
- Ordering: `Comparable`, `Comparator`
- Concurrency: `Runnable`, `Callable`

Interfaces you already use in Java

- Collections: `List`, `Set`, `Map`, `Iterable`, `Iterator`
- Ordering: `Comparable`, `Comparator`
- Concurrency: `Runnable`, `Callable`
- Resource handling: `AutoCloseable` (try-with-resources)

Why Java loves interfaces

- A class can implement **multiple interfaces** (multiple roles/capabilities)

Why Java loves interfaces

- A class can implement **multiple interfaces** (multiple roles/capabilities)
- Frameworks use interfaces for callbacks ("I will call your code")

Why Java loves interfaces

- A class can implement **multiple interfaces** (multiple roles/capabilities)
- Frameworks use interfaces for callbacks ("I will call your code")
- Standard libraries are designed around interface types (program to the abstraction)

Interface vs abstract class (simple rule)

- Use an **interface** for a capability/role with many unrelated implementations.

Interface vs abstract class (simple rule)

- Use an **interface** for a capability/role with many unrelated implementations.
- Use an **abstract class** when you need shared state + shared code for closely related classes.

Interface vs abstract class (simple rule)

- Use an **interface** for a capability/role with many unrelated implementations.
- Use an **abstract class** when you need shared state + shared code for closely related classes.
- Interfaces can have `default` methods; use them carefully to keep the contract clean.

Summary

- Interfaces reduce coupling by separating *what* from *how*.

Summary

- Interfaces reduce coupling by separating *what* from *how*.
- They enable plug-in designs (strategy), test doubles (fakes/mocks), and clean architectures (repositories).

Summary

- Interfaces reduce coupling by separating *what* from *how*.
- They enable plug-in designs (strategy), test doubles (fakes/mocks), and clean architectures (repositories).
- Many Java APIs are built around interface types.

Quick practice

1. Design an `Exporter` interface with `CsvExporter` and `JsonExporter`.

Quick practice

1. Design an `Exporter` interface with `CsvExporter` and `JsonExporter`.
2. Write a tiny test that uses a `FakeNotifier` and checks the last message sent.

Quick practice

1. Design an `Exporter` interface with `CsvExporter` and `JsonExporter`.
2. Write a tiny test that uses a `FakeNotifier` and checks the last message sent.
3. Refactor any earlier program: replace direct concrete usage with an interface + two implementations.